
PROGRAMACIÓN CONCURRENTES



Trabajo Práctico Final

Integrantes	DNI	Correo Institucional
Bejarano, Kevin	42814288	kevin.bejarano@mi.unc.edu.ar
Daniel, Tomas Gaston	44740237	tomas.daniel@mi.unc.edu.ar

Grupo: NoEsThreadSafe

Profesores:

- Luis Ventre
- Mauricio Ludemann
- Agustin Carranza

2025

Introducción

El presente informe documenta el diseño, la simulación y la implementación concurrente de un sistema de procesamiento de datos, utilizando como base el formalismo matemático de las Redes de Petri. El dominio del problema plantea una arquitectura de múltiples etapas donde los datos ingresan a través de una cola de arribos, acceden a un buffer mediante un bus de datos compartido y compiten por el uso de una unidad de procesamiento central para ejecutar tareas de complejidad simple, media o alta.

Para materializar este modelo matemático en una aplicación de software, se desarrolló un Monitor de Concurrencia implementado en Java. El diseño de este monitor se rige por el principio de separación de responsabilidades, manteniéndose completamente agnóstico a la topología de la red. Su función crítica es garantizar la exclusión mutua estricta y gestionar la sincronización de los hilos mediante barreras estructurales, semáforos y colas de condición, evitando anomalías clásicas de la concurrencia como condiciones de carrera (race conditions), deadlocks y esperas activas.

A la complejidad estructural se le añade la implementación de semántica temporal (Redes de Petri Temporizadas), asignando demoras específicas a los procesos sin comprometer la atomicidad lógica de los disparos dentro del monitor. Frente a los conflictos de ruteo, donde múltiples modos de procesamiento se encuentran habilitados simultáneamente, la arquitectura delega la toma de decisiones a una clase de políticas. A lo largo de este trabajo se muestran dos enfoques: una política de distribución estocástica (aleatoria) y una política de prioridad estricta, la cual exige un manejo riguroso para sostener la inanición (starvation) controlada de ciertos caminos de ejecución maximizando el rendimiento.

Finalmente, la correctitud del sistema no se asume, sino que se demuestra de forma analítica y empírica. El estado del sistema se valida de manera continua mediante el chequeo de Invariantes de Plaza en tiempo de ejecución, mientras que el análisis de trazas (logs) mediante expresiones regulares y algoritmos de reducción asegura que el 100% de los hilos de ejecución respeten sus Invariantes de Transición, garantizando un apagado limpio después de completar la carga de trabajo exigida.

Consignas Previas

1. Es necesario determinar con PIPE las propiedades de la red (deadlock, vivacidad, seguridad).
2. Indicar cuál o cuáles son los invariantes de plaza y los invariantes de transición de la red. Realizar una breve descripción de lo que representan en el modelo.
3. Realizar una tabla, con los estados del sistema.
4. Realizar una tabla, con los eventos del sistema.
5. Determinar la cantidad de hilos necesarios para la ejecución del sistema con el mayor paralelismo posible:
 - a. **Caso 1:** si el invariante de transición tiene un conflicto, con otro invariante, debe haber un hilo encargado de la ejecución de la/s transición/es anterior/es al conflicto y luego un hilo por invariante.
 - b. **Caso 2:** si el invariante de transición presenta un join, con otro invariante de transición, luego del join debe haber tantos hilos, como tokens simultáneos en la plaza, encargados de las transiciones restantes dado que hay un solo camino.
6. Hacer el diagrama de clases que modele el sistema.
7. Hacer el diagrama de secuencia que muestre el disparo exitoso de una transición que esté sensibilizada, mostrando el uso de la política.

Desarrollo

Propiedades de la Red

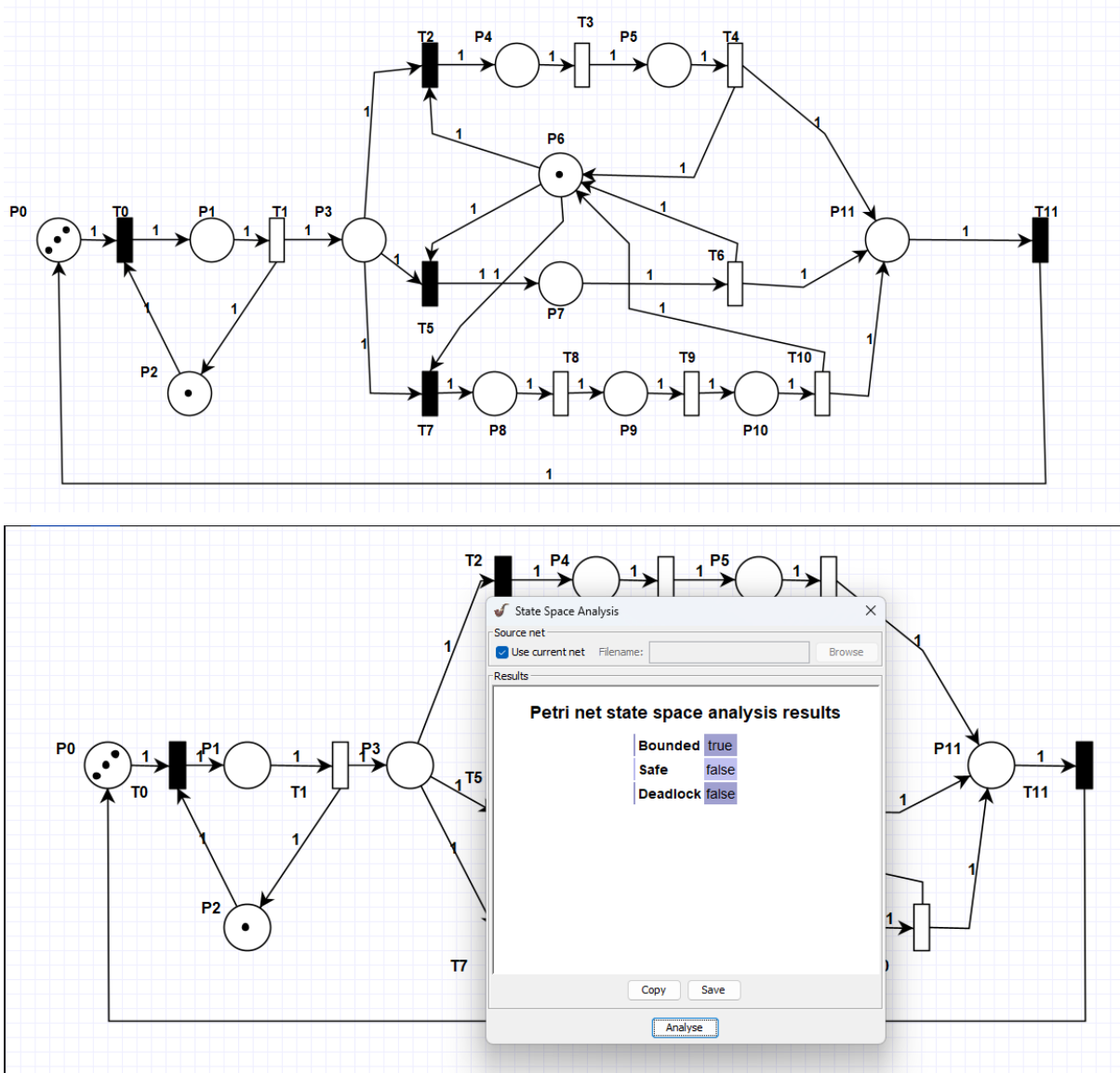
Solo para recordar, vamos a volver a definir algunos conceptos:

- **Deadlock:** Es un estado "ciego" o terminal de la red. Ocurre cuando el sistema evoluciona hacia un marcado desde el cual es estructuralmente imposible volver a sensibilizar y disparar alguna transición. Esto significa que los hilos de ejecución quedan bloqueados de forma permanente, típicamente por una dependencia circular de recursos, esperando señales o liberación de candados que nunca van a ocurrir, paralizando el sistema por completo.
- **Vivacidad:** Es la garantía analítica de que el sistema nunca se estancará de forma irremediable. Significa que, sin importar en qué estado se encuentre la red en un momento dado, siempre existe un camino que permite volver a habilitar cualquier transición.
- **Red Acotada:** Se da cuando la acumulación de tokens en cualquier plaza del modelo jamás supera un límite finito máximo, conocido como k . Esto nos garantiza que los buffers y las colas de espera no crecerán infinitamente, evitando así fallas por agotamiento de memoria
- **Red Segura:** Es el caso más estricto de una red acotada, donde el límite máximo de tokens que puede alojar cualquier plaza es exactamente 1. Conceptualmente, las redes seguras son ideales para modelar estados binarios lógicos y recursos de acceso exclusivo (Mutex libre/ocupado).

Luego de aplicar un análisis con el software PIPE, podemos determinar las siguientes propiedades de la red:

- Es una red sin deadlock
- No es una red segura. Dado que plazas como P0 o P3 están diseñadas para alojar múltiples tokens (hasta 3) simultáneamente, modelando colas de procesamiento.
- Es una red acotada con $k=3$
- Es una red Viva

Se muestra la red de petri que tenemos que analizar y la respuesta del software.



Invariantes de Plaza

Los invariantes de plaza representan leyes de conservación estructural dentro de la red. Son conjuntos de plazas donde la suma ponderada de sus tokens permanece constante ante cualquier secuencia de disparos de transición, permitiendo verificar matemáticamente la integridad de los recursos compartidos y el flujo de datos.

En nuestro modelo, se identifican tres invariantes de plaza y sus ecuaciones son:

- $M(P1) + M(P2) = 1$

Esta ecuación garantiza la exclusión mutua en el bus de datos. Asegura que solo un dato pueda estar transitando desde la entrada hacia el buffer (P3) en un instante dado.

- $M(P0) + M(P1) + M(P3) + M(P4) + M(P5) + M(P7) + M(P8) + M(P9) + M(P10) + M(P11) = 3$
Esta ecuación representa la conservación de la carga de trabajo. Demuestra que en todo momento existen exactamente 3 paquetes de datos circulando o en espera dentro del sistema, evitando pérdidas o duplicaciones de información.
- $M(P10) + M(P4) + M(P5) + M(P6) + M(P7) + M(P8) + M(P9) = 1$
Esta ecuación garantiza la exclusión mutua en la CPU. Asegura que la unidad de procesamiento central (P6) atienda un único paquete a la vez, ya sea que esté inactiva (P6) o ejecutando cualquiera de las etapas de los modos simple, medio o alto.

Invariantes de transición

Los invariantes de transición representan ciclos de vida o procesos completos dentro del sistema. Son secuencias de disparos que, al completarse, devuelven la red a su marcado inicial, confirmando que el sistema es capaz de procesar tareas de principio a fin y recuperar su estado de reposo.

Y las invariantes de transición de nuestra red son:

- $T0 \rightarrow T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T11$
Describe el ciclo de procesamiento de **Modo Medio**. Incluye el arribo del dato, su paso por el bus al buffer y la ejecución secuencial de las dos etapas de procesamiento en P4 y P5, finalizando con la descarga en el buffer de salida.
- $T0 \rightarrow T1 \rightarrow T5 \rightarrow T6 \rightarrow T11$
Describe el ciclo de procesamiento de **Modo Simple**. Representa la ruta de ejecución más directa, donde el dato es procesado íntegramente en una única etapa (P7) antes de ser liberado del sistema.
- $T0 \rightarrow T1 \rightarrow T7 \rightarrow T8 \rightarrow T9 \rightarrow T10 \rightarrow T11$
Describe el ciclo de procesamiento de **Modo Alta**. Modela la trayectoria del dato a través de las tres etapas de procesamiento consecutivas (P8, P9 y P10), capturando la secuencia de mayor carga computacional antes de retornar al marcado inicial

La red de petri modelada representa un sistema de procesamiento de datos con recursos compartidos (los que se encuentran en P2 y P6). Los datos entran a una cola (P0), pasan a un buffer de entrada (P3) mediante un bus compartido (P2), y son procesados por una unidad compartida (P6) en tres modos:

- simple (P7)
- medio (P4-P5)
- alto (P8-P9-P10)

Estados del sistema

A continuación se realiza una **tabla con los estados del sistema**, es decir que significa para nosotros cada una de las plazas de la red.

Plaza	Descripción de Estado
P0	Es una cola de arribos. Representa los paquetes de datos o tareas pendientes que están listos para ingresar al sistema.
P1	Es un estado transitorio donde un dato arribó y se encuentra esperando que el bus de acceso se libere para poder ingresar al buffer.
P2	Es un recurso compartido que actúa como un mutex. Su estado define si el bus de acceso al buffer principal está libre/ocupado, garantizando exclusión mutua en la transferencia.
P3	Es el buffer principal. Actúa como una cola de los datos que ya atravesaron el bus y están listos para ser procesados.
P4	Estado que indica que un dato está siendo procesado bajo el Modo Medio en su primera etapa.
P5	Estado que indica que el dato ha avanzado a la segunda y última etapa de procesamiento del Modo Medio.
P6	Es el recurso compartido entre los tres modos. Su estado indica si la CPU está ociosa (con token) o ejecutando alguna tarea (sin token).
P7	Estado que indica que un dato está siendo procesado bajo el Modo Simple
P8	Estado que indica que un dato está siendo procesado bajo el Modo Alto en su primera etapa.
P9	Estado que indica el avance del dato hacia la segunda etapa del Modo Alto.
P10	Estado final de procesamiento del Modo Alto antes de liberar la CPU.
P11	Es un buffer de salida. Es una cola donde se depositan los datos que ya han completado su procesamiento (indistintamente del modo) y esperan ser extraídos del sistema.

Eventos del sistema

A continuación se realiza una **tabla con los eventos del sistema**, es decir que significa para nosotros cada una de las transiciones de la red.

Transición	Descripción del Evento / Acción
T0	Un nuevo paquete de datos ingresa al sistema y se posiciona a la espera del bus de transferencia.
T1	El sistema adquiere el bus de datos (P2), transfiere el paquete al buffer principal (P3) y libera el bus de forma atómica.
T2	Inicio Modo Medio
T3	El proceso de complejidad media finaliza su primera etapa y avanza a la segunda.
T4	Fin Modo Medio. Libera la CPU (P6) para otros procesos y deposita el resultado en el buffer de salida.
T5	Inicio Modo Simple
T6	Fin Modo Simple. Libera la CPU (P6) y deposita el resultado en el buffer de salida.
T7	Inicio Modo Alto
T8	El proceso de alta complejidad avanza de su primera a su segunda etapa
T9	El proceso de alta complejidad avanza a su tercera y última etapa
T10	Fin Modo Alto. Libera la CPU (P6) y deposita el resultado final en el buffer de salida.
T11	Salida del sistema. El sistema extrae un dato ya procesado del buffer de salida, finalizando su ciclo de vida y reponiendo la capacidad inicial de carga.

Cantidad de hilos necesarios para maximizar el paralelismo

Para realizar este punto, debemos basarnos en la información que nos brinda el Paper de [Algoritmo para determinar Cantidad y responsabilidad de hilos en sistemas embebidos modelados con redes de petri](#). En el cual se indica que tenemos que seguir los pasos de la sección de **Algoritmo para la determinación de hilos máximos activos simultáneos**.

Previo a la aplicación, identificamos las plazas especiales de nuestra red S3PR (igual que hace el paper):

- **Plazas idle:** idle = {P0}
- **Recursos:** recursos = {P6} (Unidad de procesamiento)
- **Restricciones:** restricciones = {P2} (Bus de acceso)

1.1. Obtener los IT de la RdP. En nuestro caso son los siguientes:

$$IT_1 = \{T0, T1, T2, T3, T4, T11\}$$

$$IT_2 = \{T0, T1, T5, T6, T11\}$$

$$IT_3 = \{T0, T1, T7, T8, T9, T10, T11\}$$

1.2. Obtenemos el conjunto de plazas asociadas PI_i de cada IT:

$$PI_1 = \{P0, P1, P2, P3, P4, P5, P6, P11\}$$

$$PI_2 = \{P0, P1, P2, P3, P6, P7, P11\}$$

$$PI_3 = \{P0, P1, P2, P3, P6, P8, P9, P10, P11\}$$

1.3. Obtenemos el conjunto de plazas de acción PA_i de cada IT.

Para hacer esto, restamos de cada conjunto PI_i las plazas idle, recursos y restricciones ({P0, P2, P6}):

$$PA_1 = \{P1, P3, P4, P5, P11\}$$

$$PA_2 = \{P1, P3, P7, P11\}$$

$$PA_3 = \{P1, P3, P8, P9, P10, P11\}$$

1.4. Obtenemos el conjunto de estados MA del conjunto de plazas PA .

Donde la unión de todas las plazas de acción es:

$$PA = \{P1, P3, P4, P5, P7, P8, P9, P10, P11\}$$

Al observar todos los marcados posibles generados por el árbol de alcanzabilidad para este conjunto PA , la suma máxima de tokens presentes simultáneamente en estas plazas es 3. Esto está directamente dictado por el invariante de plaza que establece la capacidad máxima del sistema (los 3 tokens inyectados por P0). Por lo tanto, el marcado máximo global indica un límite de 3 hilos activos simultáneos para el flujo total de datos. A continuación, aplicamos el algoritmo de la sección 4.2 (del paper) para determinar los segmentos de ejecución y responsabilidades de cada hilo:

1. Se observa en la red que los tres invariantes de transición comparten un camino inicial (T0, T1) y presentan un conflicto (fork) en la plaza P3. Esto cumple con la condición de bifurcación, determinando el siguiente segmento previo al conflicto:
 - **Segmento previo al fork:** Segmento S_A y plazas $PS_A = \{P1, P3\}$
2. A partir de P3, los invariantes se dividen en tres caminos independientes, definiendo los siguientes segmentos de ejecución:
 - **Segmento Superior (Medio):** Segmento S_B y plazas $PS_B = \{P4, P5\}$
 - **Segmento Central (Simple):** Segmento S_C y plazas $PS_C = \{P7\}$
 - **Segmento Inferior (Alto):** Segmento S_D y plazas $PS_D = \{P8, P9, P10\}$
3. Finalmente, se observa que los tres invariantes cumplen el **caso 3** del algoritmo al converger mediante una unión (join) en la plaza P11 para disparar T11, definiendo el último segmento:
 - **Segmento post-join:** Segmento S_E y plazas $PS_E = \{P7\}$

Una vez determinados los segmentos, aplicamos el algoritmo de la sección 4.3 (del paper) para obtener los hilos máximos necesarios por cada segmento de ejecución, calculando el marcado máximo $Max(MS_i)$ de sus respectivas plazas de acción:

- $Max(MS_A) = 3$ (El buffer P3 y P1 pueden alojar hasta 3 tokens provenientes de P0).
- $Max(MS_B) = 1$ (Limitado por la exclusión mutua del recurso P6).
- $Max(MS_C) = 1$ (Limitado por la exclusión mutua del recurso P6).
- $Max(MS_D) = 1$ (Limitado por la exclusión mutua del recurso P6).
- $Max(MS_E) = 3$ (El buffer de salida P11 puede llegar a acumular los 3 tokens del sistema esperando salir).

El algoritmo de la sección 4.3 dice explícitamente que la cantidad de hilos para un segmento se determina buscando el marcado máximo:

$$Max(MS_i)$$

posible en las plazas de ese segmento. El "Segmento extra" (Segmento E) contiene a la plaza de acción P11.

Entonces, cuál es el máximo número de tokens que pueden estar simultáneamente en P11? Si la CPU procesó rápido y la salida se demoró, los 3 paquetes del sistema pueden acumularse ahí. Por lo tanto, $Max(MS_E) = 3$.

La teoría nos dice que para ese 1 segmento extra, el paralelismo máximo exige 3 hilos.

Una vez determinados los hilos máximos por segmento, la suma de todos estos determina la máxima cantidad de hilos teóricos del sistema bajo la metodología estructural. Para nuestra red, la suma $(3 + 1 + 1 + 1 + 3)$ es igual a 9 hilos.

No obstante, en la implementación práctica, decidimos optimizar la arquitectura delegando este segmento a 1 único hilo recolector real al igual que los 3 hilos de entrada debido a que no había una mejora en el rendimiento

A continuación se muestra el gráfico que indica la cantidad de hilos mínimos para maximizar el paralelismo de la red.

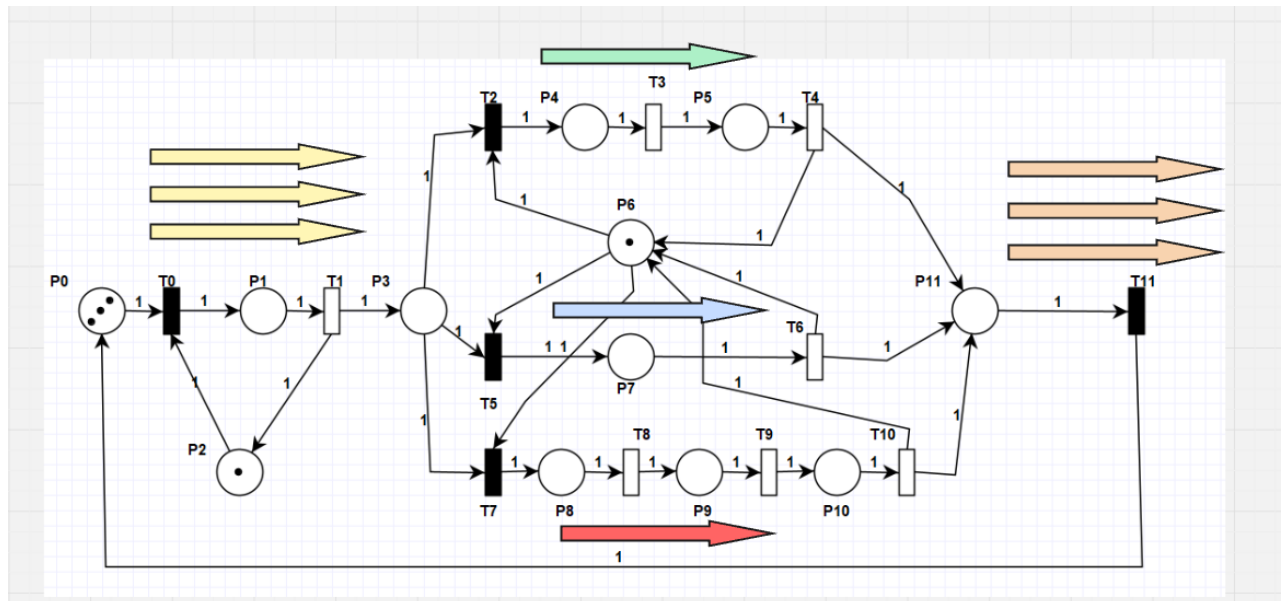


Diagrama de Clases

El diagrama de clases que modela al problema es el siguiente:

Diagrama de Secuencias de un disparo exitoso

El diagrama de secuencias de un disparo exitoso es el siguiente:

Políticas

Analizando la topología de la red, se identifica un conflicto estructural crítico en la plaza P3 (buffer de datos a procesar). Al arribar un token a esta plaza, el sistema se enfrenta a una bifurcación (fork) donde tres transiciones independientes (T2, T5 y T7) compiten simultáneamente por el recurso compartido de la CPU (P6). Dado que un mismo paquete de datos no puede ser procesado por más de un modo a la vez, el monitor de concurrencia requiere un mecanismo de decisión externo para determinar qué hilo debe adquirir el mutex y avanzar.

Para resolver este conflicto tomamos la decisión de utilizar una interfaz en lugar de codificar la lógica condicional directamente dentro del Monitor, esto lo hacemos por dos grandes motivos:

- **Separación de Responsabilidades:** El Monitor debe ser completamente agnóstico respecto a la lógica de negocio. Su única responsabilidad es garantizar la exclusión mutua y la sincronización de los hilos. No debe saber "quién" tiene prioridad.
- **Principio Abierto/Cerrado:** Al inyectar la política como una dependencia en el constructor del Monitor, el sistema queda abierto a la extensión (se pueden agregar nuevas políticas en el futuro) pero cerrado a la modificación.

Un detalle crítico en la implementación del Monitor es cómo se alimenta a la política. El Monitor no le pasa a la interfaz las transiciones puramente "sensibilizadas" por la matemática de la red, ya que esto podría causar un escenario de livelock o señales perdidas (Lost Signals). Antes de consultar a la política, el Monitor cruza los datos de la Red de Petri con los de la Cola de Condición (`colaCondicion.hasEnqueued(i)`). De esta manera, la política recibe un arreglo booleano únicamente con las transiciones que están habilitadas y que además tienen un hilo real esperando para ejecutarlas.

En base a los requerimientos del sistema, se desarrollaron dos implementaciones concretas de esta interfaz:

- **Política Aleatoria:** esta implementación modela un comportamiento estocástico y justo (fairness). Al recibir el arreglo de transiciones posibles, recopila los índices válidos en una estructura dinámica y utiliza la clase genérica `Random` para seleccionar un ganador con probabilidad uniforme. Esta política garantiza que no exista inanición (starvation) intencional, permitiendo que los modos de procesamiento simple, medio y alto se distribuyan de forma natural según los tiempos de bloqueo temporal de cada ruta.
- **Política Prioritaria:** esta implementación introduce un esquema de inanición controlada. Cumpliendo con la restricción de maximizar el rendimiento, la política prioriza estrictamente el modo de complejidad simple (T5). El algoritmo interno evalúa el arreglo de posibilidades secuencialmente buscando primero el índice correspondiente a T5. Si este camino es posible, la política lo elige absoluta e indefectiblemente, obligando a los hilos del modo medio y alto a ceder el acceso y retornar a la cola de condición. La CPU solo es cedida a los modos más complejos durante las ventanas de tiempo en las que el hilo simple se encuentra suspendido (`Thread.sleep()`)

LOG

Para garantizar una verificación exhaustiva y no intrusiva del sistema, se implementó una arquitectura de registro basada en dos archivos de *log* independientes. Esta separación de responsabilidades permite auditar tanto la correctitud estructural de la Red de Petri como el comportamiento temporal de los hilos a nivel del sistema operativo.

Log de Transiciones

Este archivo se compone exclusivamente de la secuencia "cruda" de transiciones disparadas exitosamente (T0-T1-T5-T6-T11...). Su objetivo es indicar la evolución del marcado de la red.

Este archivo es el insumo directo para el análisis mediante Expresiones Regulares (Regex). Las expresiones regulares actúan como un algoritmo de reducción que buscan los patrones exactos de los Invariantes de Transición y los eliminan del texto. Si al finalizar la lectura el archivo este queda reducido a una cadena completamente vacía (sin caracteres sobrantes), se demuestra empíricamente que el 100% de los disparos pertenecieron a ciclos válidos. Esto confirma la ausencia total de transiciones huérfanas, fallos de ruteo o secuencias interrumpidas.

Log de Trazabilidad Temporal

Este segundo archivo almacena marcas de tiempo exactas (timestamps) y detalla el ciclo de vida de cada hilo, en qué milisegundo intentan adquirir el monitor, cuándo liberan el mutex para ir a dormir (`Thread.sleep()`), y en qué orden son despertados por las colas de condición.

El propósito fundamental de este log es demostrar el correcto funcionamiento de las barreras de sincronización y las políticas. Al analizar las marcas de tiempo, podemos verificar comportamientos de la concurrencia que no son visibles en la matriz matemática.

Algo muy útil de este LOG es que nos permite comprobar que el sistema operativo respeta el orden de llegada al monitor. Por ejemplo, si el Hilo A ingresa al monitor, verifica que le falta tiempo, suelta el mutex y se va a dormir; y posteriormente un Hilo B ingresa, hace lo mismo pero con un tiempo de sueño mucho menor, el log nos permitirá visualizar el comportamiento que esperamos que es que aunque el Hilo B despierte antes, el sistema no le permite "robar" la ejecución. El log demostrará que B queda bloqueado intentando re-adquirir el mutex, teniendo que esperar obligatoriamente a que el Hilo A (que llegó primero a la cola de readquisición) termine su tarea. Esto prueba que nuestro diseño del Monitor evita que hilos rápidos generen inanición sobre hilos más lentos.

Análisis Temporal

Para cumplir con lo pedido en la consigna, implementamos la semántica temporal asignando valores alfa a las transiciones temporales de nuestra red. Recordando que las transiciones temporales son {T1, T3, T4, T6, T8, T9, T10} y las otras transiciones que no están aquí son instantáneas es decir que las consideramos como alfa=0. Los valores que elegimos fueron

pensados para que el programa se ejecute dentro del rango solicitado que sería de 20 a 40 segundos.

Los valores de alfa asignados son los siguientes:

Transición	Alfa (ms)
T1	100
T3	100
T4	100
T6	100
T8	100
T9	100
T10	100

Decidimos usar el mismo valor para todas las transiciones temporales porque no hay un motivo por el que deberían ser distintos y además el hecho de que sean iguales nos simplifica el análisis.

Analisis teorico de los tiempos

Para saber si el valor de 100 ms es correcto, primero analizamos los tres caminos posibles que puede tomar un dato dentro del sistema y sumamos sus tiempos alfa. Nuevamente recordamos que las transiciones {T0, T2, T5, T7 y T11} son instantáneas así que no suman tiempo de espera.

Invariante	Camino	Sumatoria total de tiempo
Simple	T0 -> T1 -> T5 -> T6 -> T11	$T1+T6 = 200 \text{ ms}$
Medio	T0 -> T1 -> T2 -> T3 -> T4 -> T11	$T1+T3+T4 = 300 \text{ ms}$
Alto	T0 -> T1 -> T7 -> T8 -> T9 -> T10 -> T11	$T1+T8+T9+T10 = 400 \text{ ms}$

Ahora si ejecutamos las 200 invariantes por completo en serie (sin tener en cuenta el paralelismo) podemos calcular los tiempos extremos asumiendo que todas van por el mismo camino, lo que nos da tres posibilidades:

1. **Caso optimista (todo simple):** $200 * 200 \text{ ms} = 40 \text{ segundos}$
2. **Caso intermedio (todo medio):** $200 * 300 \text{ ms} = 60 \text{ segundos}$
3. **Caso pesimista (todo alto):** $200 * 400 \text{ ms} = 80 \text{ segundos}$

Como podemos observar, estos valores en serie en principio no nos sirven ya que en el caso más pesimista estamos muy por encima del límite de los 40 segundos. Sin embargo, aquí es donde entra el paralelismo que nos mejora considerablemente los tiempos de ejecución. Nuestra red tiene 3 tokens en el sistema y los recursos compartidos que son P2 (bus) y P6 (cpu) son independientes entre sí. Esto significa que mientras un dato está usando la CPU, otro dato puede estar usando el bus. Esto genera el efecto de “pipeline” (lo vemos en otras materias) que es cuando varios datos se procesan al mismo tiempo en diferentes etapas.

Si analizamos el tiempo que cada dato ocupa la CPU (recurso P6) que es el cuello de botella del sistema tenemos que:

- **Modo simple:** el tiempo de CPU por invariante completada es de 100 ms (T6)
- **Modo medio:** el tiempo de CPU por invariante completada es de 200 ms (T3 + T4)
- **Modo alto:** el tiempo de CPU por invariante completada es de 400 ms (T8 + T9 + T10)

El bus P2 permite inyectar un dato cada 100 ms (es la duración de T1) y la CPU los procesa a su ritmo dependiendo del camino. Con 3 tokens en el sistema, el pipeline se mantiene saturado y el tiempo total depende de cuánto tiempo pasa cada dato en la CPU. Es importante entender que llegamos a esta conclusión porque el tiempo que permanece el dato en la CPU es el tiempo mayor, el BUS demora menos y como se ejecutan en paralelo este tiempo no es tenido en cuenta ya que no es el dominante.

Para calcular el tiempo teórico con esta pipeline, necesitamos saber cómo se distribuyen los invariantes en los tres caminos. Esta distribución depende de la política utilizada. Hicimos 10 ejecuciones con cada política para sacar los porcentajes promedio:

Política Prioritaria

Iteración	Simple (invariantes)	Medio (invariantes)	Alto (invariantes)
1	100 (50%)	45 (22.5%)	55 (27.5%)
2	100 (50%)	48 (24.0%)	52 (26.0%)
3	100 (50%)	49 (24.5%)	51 (25.5%)
4	100 (50%)	42 (21.0%)	58 (29.0%)
5	100 (50%)	48 (24.0%)	52 (26.0%)
6	100 (50%)	58 (29.0%)	42 (21.0%)
7	101 (50.5%)	52 (26.0%)	47 (23.5%)
8	100 (50%)	49 (24.5%)	51 (25.5%)
9	100 (50%)	43 (21.5%)	57 (28.5%)

10	100 (50%)	54 (27.0%)	46 (23.0%)
Promedio	100.1 (50.05%)	48.8 (24.4%)	51.1 (25.55%)

Se puede ver que con la política prioritaria, aproximadamente la mitad de los invariantes van por el camino simple mientras que el medio y el alto se reparten la otra mitad. Esto claramente no es casualidad sino que es una consecuencia directa de cómo funciona la política y la estructura de la red.

Cuando el camino simple termina (T6 dispara y libera P6), el hilo simple todavía está dentro del método fire Transition terminando su ejecución, osea que todavía no volvió a pedir T5. En ese instante, el monitor revisa la condición (Vs AND Vc) y ve que T5 no está en Vc, pero que T2 y T7 si. Si P3 tiene tokens, la política elige entre medio o alta para la siguiente ronda. Luego el hilo de simple vuelve a la cola y gana las próximas porque es prioritario hasta que vuelve a terminar y el ciclo se repite.

Esto quiere decir que el flujo va a ser algo así:

1. HILO SIMPLE GANÓ, dispara T6 y libera P6.
2. HILO ALTO O MEDIO GANÓ, dispara su respectiva transición al final y libera P6.
3. HILO SIMPLE GANÓ
4. HILO ALTO O MEDIO GANÓ

Así que siempre de cada 2 iteraciones el dato va a elegir pasar por el camino simple, es por esto que el hilo simple se queda con el 50% de que el dato elige pasar por él mientras que los otros 2 comparten el porcentaje que sobra.

Política Aleatoria

Iteración	Simple (invariantes)	Medio (invariantes)	Alto (invariantes)
1	67 (33.5%)	67 (33.5%)	66 (33.0%)
2	60 (30.0%)	70 (35.0%)	70 (35.0%)
3	65 (32.5%)	73 (36.5%)	62 (31.0%)
4	65 (32.5%)	67 (33.5%)	68 (34.0%)
5	63 (31.5%)	65 (32.5%)	72 (36.0%)
6	70 (35.0%)	65 (32.5%)	65 (32.5%)
7	64 (32.0%)	69 (34.5%)	67 (33.5%)
8	71 (35.5%)	57 (28.5%)	72 (36.0%)
9	64 (32.0%)	75 (37.5%)	61 (30.5%)
10	67 (33.5%)	62 (31.0%)	71 (35.5%)
Promedio	65.6 (32.8%)	67.0 (33.5%)	67.4 (33.7%)

Con la política aleatoria los tres caminos se distribuyen de forma pareja (aproximadamente 33% cada uno) que es justo lo que esperamos de una selección aleatoria con probabilidad uniforme.

Con estos porcentajes, podemos calcular el tiempo de CPU promedio por invariante y al multiplicarlo por el total de invariantes obtenemos el tiempo total de ejecución.

- **Con política prioritaria (50% simple, 25% medio, 25% alto):** el tiempo de CPU promedio se calcula de la siguiente manera:

$$0.5 * 100 \text{ ms} + 0.25 * 200 \text{ ms} + 0.25 * 300 \text{ ms} \rightarrow 175 \text{ ms} * 200 (\text{invariantes}) = 35 \text{ seg}$$

- **Con política aleatoria (33 % cada uno):** el tiempo de CPU promedio se calcula de la siguiente manera:

$$0.33 * 100 \text{ ms} + 0.33 * 200 \text{ ms} + 0.33 * 300 \text{ ms} \rightarrow 198 \text{ ms} * 200 (\text{invariantes}) \approx 40 \text{ seg}$$

Ahora para la implementación de los tiempos volvimos a ejecutar 10 veces nuestro programa para ver en promedio el tiempo de ejecución dependiendo de cada política.

Iteración	Política Prioritaria (ms)	Política Aleatoria (ms)
1	35808	40216
2	35499	41318
3	35418	40016
4	36104	40617
5	35501	41220
6	34502	39815
7	34901	40615
8	35408	40418
9	36007	40023
10	34902	40723
Promedio	35405	40498

Como podemos observar los resultados obtenidos coinciden bastante con lo esperado teóricamente por lo que podemos confirmar que el tiempo que seleccionamos es correcto.

Conclusión

Al comenzar el desarrollo, nuestro objetivo era bastante claro, implementar un monitor de concurrencia que ejecute la Red de Petri de procesamiento de datos de forma correcta y eficiente. El modelo teórico estaba definido y los invariantes identificados, pero cuando pasamos a la práctica tuvimos varios desafíos que nos obligaron a tomar decisiones de diseño importantes.

El primer desafío fue diseñar un monitor que fuera completamente agnóstico a la red, como exige la consigna. Nuestra solución fue implementar la política Signal and Exit (SX), donde el único método público es `fireTransition()` y todas las dependencias (Mutex, ColaCondicion, Política) se inyectan por constructor. Esto permite cambiar la red sin modificar el monitor. La política SX fué muy importante para evitar señales perdidas, cuando un hilo dispara exitosamente y despierta a otro, sale del método sin liberar el mutex, y el hilo despertado lo hereda directamente (handoff). Esto elimina la necesidad de que el hilo despertado compita nuevamente por el mutex.

La semántica temporal fue otro punto crítico. Implementamos el cálculo de alfa restando el timestamp de sensibilización del tiempo actual (como lo dice la teoría). Cuando una transición todavía no cumple su tiempo, el hilo libera el mutex y duerme fuera del monitor, permitiendo que otros hilos avancen. Esto evita esperas activas y mantiene el sistema eficiente. Los valores alfa los fijamos en 100 ms para todas las transiciones temporales, basándonos en un análisis que considero el pipeline de 3 tokens y el cuello de botella de P6. Con este valor, las ejecuciones reales dieron ~35s para política prioritaria y ~40s para aleatoria, ambos dentro del rango exigido de 20 a 40 segundos.

La gestión de los recursos compartidos P2 (bus) y P6 (CPU) fue validada mediante los invariantes de plaza, que se verifican automáticamente después de cada disparo en el método `disparar()` de la clase RdP. Si un disparo violara un invariante, el programa lanza una excepción y se detiene inmediatamente. Esto nos garantiza que la red nunca alcance un estado inconsistente.

Para la verificación de invariantes de transición implementamos dos mecanismos. El primero es `AnalizadorInvariantes`, que lee el log de transiciones y cuenta las ocurrencias de T4, T6 y T10 (finales de cada camino) comparándolas con T11. El segundo es un script en Python con expresiones regulares que reduce la cadena de transiciones eliminando patrones de invariantes válidos. Si al finalizar queda texto sobrante, significa que hay disparos erróneos. Ambos métodos confirmaron que el 100% de los 200 invariantes se completaron correctamente.

El Logger genera dos archivos: `log.txt` con un historial detallado de cada evento (adquisición del mutex, disparo, sleeps, handoffs) y `log_transiciones.txt` con la secuencia cruda para el análisis con Regex. Este log nos permitió verificar visualmente el comportamiento del monitor y confirmar que el orden de los eventos era correcto.

En cuanto a las políticas, observamos que la relación 50-25-25 de la política prioritaria no es arbitraria, sino que surge naturalmente de la interacción entre el handoff y los tiempos de CPU de cada camino. Cuando el camino simple termina (T6 dispara), el hilo Simple tarda

unos microsegundos en volver a pedir T5. En ese instante, el monitor despierta a Medio o Alta que ya estaban esperando. Luego Simple vuelve a la cola, gana las siguientes rondas por ser prioritario, hasta que vuelve a terminar y el ciclo se repite. Esto genera una alternancia que da como resultado ~50% Simple, ~25% Medio y ~25% Alta.

Pudimos confirmar que:

- Los invariantes de plaza se cumplen en todas las ejecuciones, verificados automáticamente post-disparo.
- Los invariantes de transición se cumplen, verificados tanto por AnalizadorInvariantes como por el script Regex con 200 invariantes completados.
- Los recursos críticos P2 y P6 son utilizados correctamente, garantizando exclusión mutua en el bus y la CPU.
- Las políticas funcionan como esperábamos: la prioritaria da ~50% simple (~35s) y la aleatoria distribuye parejo (~33% cada uno, ~40s).
- El análisis temporal con alfa=100 ms mantiene el programa dentro del rango de 20 a 40 segundos.
- El monitor SX con handoff es correcto y coincide exactamente con el diagrama de secuencias.

En conclusión, la implementación de este sistema de procesamiento de datos basado en una Red de Petri temporizada nos permitió aplicar los conceptos de concurrencia vistos en la materia: exclusión mutua, sincronización por condición, semántica temporal, políticas de resolución de conflictos y verificación de invariantes. Cada decisión de diseño fue orientada a mantener la simplicidad del código sin sacrificar la corrección, y las herramientas de verificación implementadas nos permiten afirmar con confianza que el sistema funciona correctamente en todos los escenarios evaluados.